

SOFTWARE ENGINEERING

UNIT – 2

TOPIC – 6

INTRODUCTION TO VERSION CONTROL SYSTEM, GIT, GITHUB

Introduction to Version Control Systems (VCS)

In software development, managing and organizing code changes is essential. Projects can involve multiple developers who work on different parts of the code simultaneously. Without a way to track changes, things can quickly get confusing, and mistakes can easily happen. This is where Version Control Systems (VCS) come in.

Key Points:

- VCS tools help in **tracking, managing, and organizing code changes**.
- They **store different versions** of the code, so you can go back to previous versions if something goes wrong.
- VCS enables **multiple developers** to work on the same project, allowing easy **collaboration**.
- Example: Think of VCS like a "track changes" feature in a Word document, but for code, where you can see who made changes, when, and why.

What is a Version Control System (VCS)?

A Version Control System (VCS) is a tool that helps keep track of the changes made to files over time. It stores every version of the code, so you can revert to earlier versions if needed. It allows developers to work together without overwriting each other's changes.

Benefits of VCS:

1. **Keeps a history of changes:** Every modification is recorded, and you can see who made which change.

2. **Collaboration:** Multiple developers can work on the same project simultaneously without conflicts.
3. **Easy recovery:** If a mistake is made, you can go back to a previous version without losing any work.

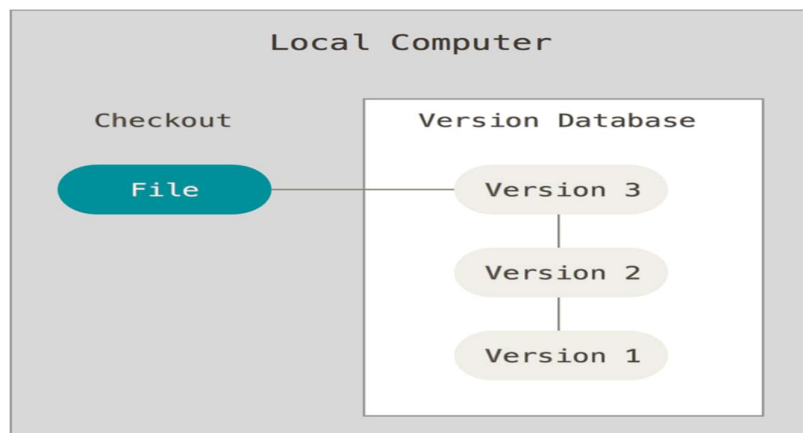
Example: Imagine you're writing a report, and every time you save it, a backup copy is created. You can always go back to an earlier version if the new changes don't work out.

Types of Version Control Systems

There are three main types of VCS, each suited to different situations:

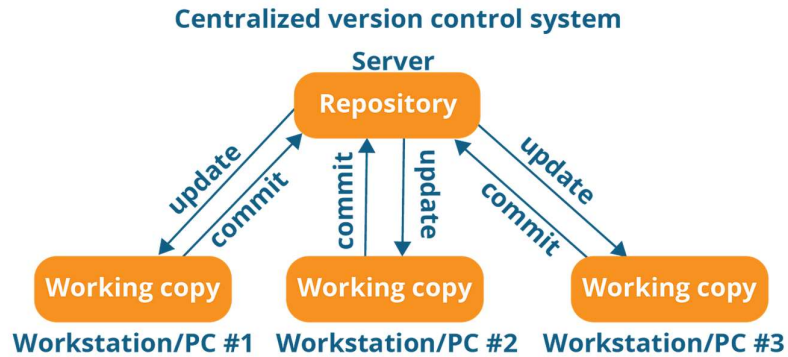
1. Local Version Control System:

- Stores versions on your local computer.
- **Problem:** If your computer crashes, you lose everything.
- **Example:** Saving different versions of a file on your desktop.



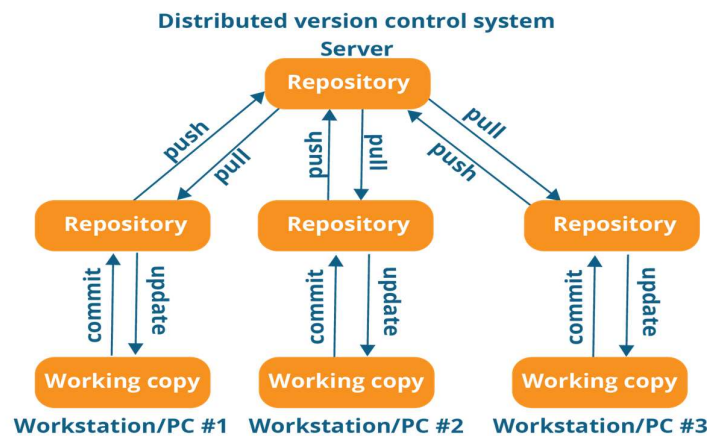
2. Centralized Version Control System:

- All versions are stored on a central server. Users connect to this server to access the files.
- **Problem:** If the server crashes, you could lose all your work.
- **Example:** Like a shared folder on Google Drive that everyone can edit.



3. Distributed Version Control System (DVCS):

- Each user has a complete copy of the project, including its entire history.
- **Advantage:** Even if the main server fails, copies exist on multiple devices, ensuring data safety.
- **Example:** Think of it like everyone on the team having their own complete backup of a shared project folder.



Why Distributed VCS is Better:

- Provides **more safety** because data is stored in multiple places.
- Allows **better collaboration** since everyone has a complete copy, making it easy to work offline.

Introduction to Git

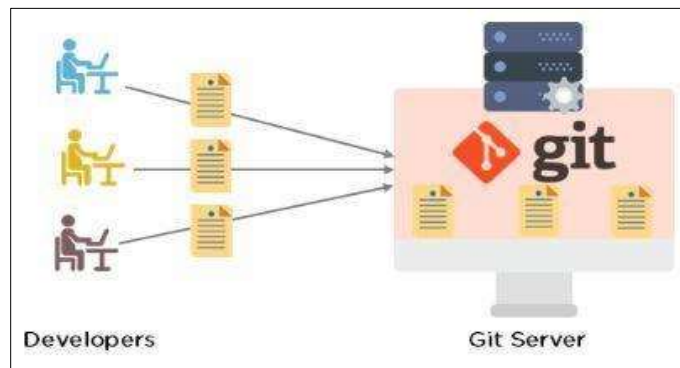
As projects grow larger and involve more people, managing changes can become difficult. This is where **Git**, a Distributed VCS, comes in.

What is Git?

- **Git** is a tool that tracks changes in files, especially code, over time.
- It was created by **Linus Torvalds** in **2005** to manage the development of the Linux kernel.
- Git helps multiple people work on the same project simultaneously without causing conflicts.

Features of Git:

1. **Free and open-source:** Anyone can use Git without cost.
2. **Supports collaboration:** Multiple developers can work on the same project at once.
3. **Non-linear development:** Git allows you to create and manage multiple branches of a project.
4. **Tracks entire history:** Every change is saved, so you can go back to any version.



Why Git is Popular:

- Efficiently handles large projects with many contributors.
- Allows for **branching**, where developers can create separate copies of the code to work on features without affecting the main codebase.

How Git Works (Git Workflow)

To use Git effectively, it's important to understand its workflow, which involves three main stages:

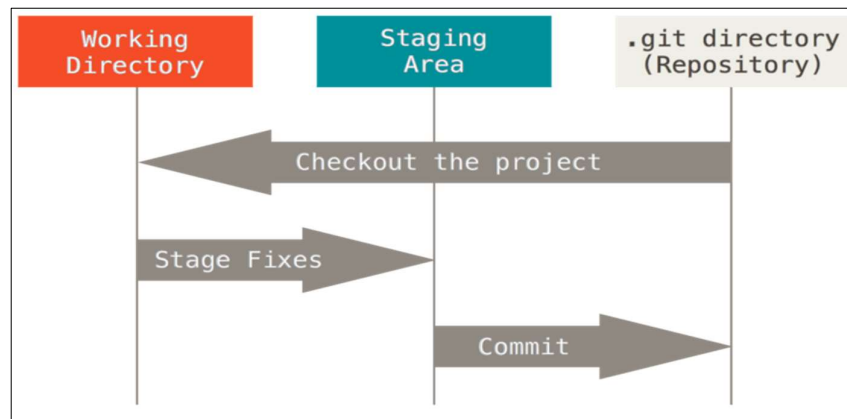
1. **Working Directory:**
 - This is where you make changes to your files.
 - **Example:** Editing a file on your computer.

2. Staging Area:

- This is where you prepare selected changes to be saved permanently.
- **Example:** Think of it as placing files in a "to-do list" before adding them to your final project.

3. Git Repository:

- This is where all your changes are stored, including the entire history of your project.
- **Example:** A folder containing all your backups and old versions of the files.



Steps in Git Workflow:

1. **Edit files** in the Working Directory.
2. **Stage** the files you want to commit (save permanently) in the Staging Area.
3. **Commit** the changes to the Git Repository, where they are saved permanently.

Transition to GitHub: Why Do We Need It?

While Git is great for tracking changes, there's a need for **sharing and collaborating on code** across different locations. This is where **GitHub** comes in.

Problem with Just Git:

- If you're working on a project with people who are not in the same place, it's difficult to share changes and keep everything up-to-date.
- GitHub solves this by providing a **platform** where you can **store your Git projects online**, making it easier for teams to collaborate.

What is GitHub?

GitHub is a **web-based platform** that hosts Git repositories, making it easier for teams to **share, collaborate, and manage** their code projects.

Key Points:

1. **Definition:** GitHub is an online service where developers can store, manage, and share their Git projects.
2. **Purpose:** Allows teams to work on the same project from different locations, making collaboration easy.
3. **Owned by:** Microsoft since 2018.
4. **Example:** Imagine GitHub as a social network for programmers where they can share and discuss their code.

Features of GitHub

GitHub offers many features that make project management and collaboration more efficient:

1. **Project Management:** Helps teams track tasks, assign work, and organize projects.
 - **Example:** Like a to-do list that everyone can see and update.
2. **Secure Code Packages:** Safely store and share pieces of code with your team or publicly.
 - **Example:** Uploading a useful function that others can use.
3. **Team Collaboration:** Tools to communicate, review, and manage code changes.
 - **Example:** Commenting on a teammate's code to suggest improvements.
4. **Pull Requests (Code Review):** Discuss and review code changes before merging them into the main project.
 - **Example:** Like submitting a draft for approval before it's published.
5. **Security Tools:** Detect and fix weaknesses in your code.
 - **Example:** Checking for errors before releasing a product.
6. **Code Hosting:** Millions of repositories hosted, with easy access to project details and history.

Alternatives to GitHub

While GitHub is the most popular platform for hosting Git projects, there are other services that provide similar features:

1. **GitLab:** A complete platform for DevOps and collaboration, offering tools for project management and CI/CD.
2. **Bitbucket:** Often used by smaller teams; integrates well with Atlassian tools like Jira.
3. **AWS Code Commit:** Managed Git hosting provided by Amazon Web Services.
4. **SourceForge:** Known for hosting open-source software projects.

Differences Between Git and GitHub

While Git and GitHub are used together, they serve different purposes. Here's how they compare:

Aspect	Git	GitHub
Type	Software tool for tracking code changes	Online platform for hosting Git projects
Interface	Works through the command line	User-friendly graphical web interface
Location	Installed on your local computer	Accessible through a web browser
Purpose	Manages versions and history of code	Helps in sharing, collaborating, and managing projects
Ownership	Developed by Linus Torvalds	Owned by Microsoft
Focus	Version control and tracking code changes	Collaboration, project management, and code sharing
Example	A tool to save and track changes in your project files	A website where you can share your Git projects with others

Git: An essential tool that tracks changes, manages versions, and helps developers work on code without conflicts.

GitHub: A platform that allows teams to share and collaborate on Git projects from anywhere in the world.

SOFTWARE ENGINEERING

UNIT – 2

TOPIC – 7

BASIC GIT COMMANDS - VERSION, CONFIG, INIT, STATUS, ADD, COMMIT, DIFF, HELP

Basic Git Commands

1. Git Version:

The **git --version** command is used to check if Git is installed on your system. This command displays the current version of Git that is available.

For example, running: **git --version**

might return `git version 2.37.1`, confirming that Git is installed.



```
MINGW64/c/Users/kmit
kmit@DESKTOP-50B7B83 MINGW64 ~
$ git --version
git version 2.37.1.windows.1
```

Knowing the installed Git version helps in troubleshooting and ensuring compatibility with certain Git features or integrations. It is the first step before using Git functionalities.

2. Git Config:

The **git config** command is used to set up the user identity for commits, including name and email address. This ensures that every commit you make in the project history is associated with your correct identity.

Usage Example:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Explanation:

- **--global:** Applies the configuration globally for the system, meaning it applies to all projects.
- **user.name** and **user.email:** These options specify the identity that will appear in the commit history.

```
kmit@DESKTOP-50B7B83 MINGW64 ~  
$ git config --global user.name "savram674"  
  
kmit@DESKTOP-50B7B83 MINGW64 ~  
$ git config --global user.email "savitharamesh674@gmail.com"  
  
kmit@DESKTOP-50B7B83 MINGW64 ~
```

Example Output:

```
git config --global user.name "Madhurika"  
git config --global user.email "madhurika.kmit@example.com"
```

In this example, Jane Doe's details will be associated with all commits made on this system unless overridden for a specific repository.

3. Git Init:

The **git init** command is used to create a new Git repository. This initializes a directory as a new Git repository and creates a **.git** subdirectory to store all Git-related files.

Usage Example: git init

Explanation: This command creates an empty Git project within the directory. After running **git init**, the project is now under version control, and you can start tracking changes.

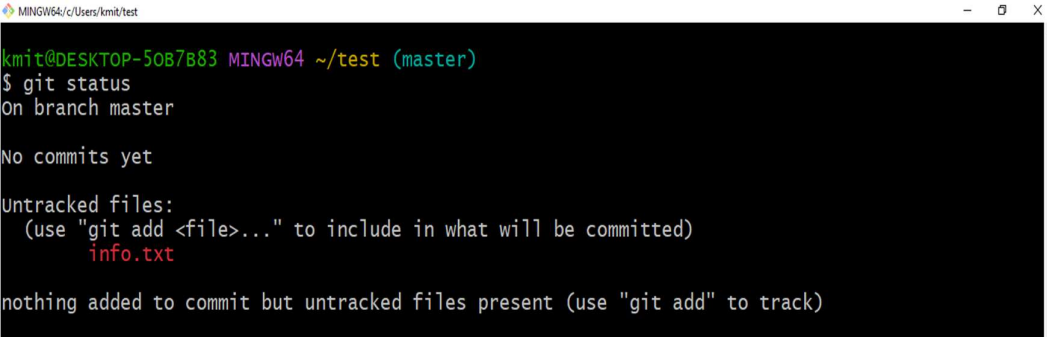
```
kmit@DESKTOP-50B7B83 MINGW64 ~/test  
$ git init  
Initialized empty Git repository in C:/Users/kmit/test/.git/
```

Real-World Example: Imagine you have a directory called **my-project**. Running **git init** inside it will turn it into a Git repository, allowing you to track its changes.

4. Git Status:

The **git status** command displays the state of your working directory and staging area. It shows what changes have been staged, what files are not being tracked by Git, and what changes are yet to be committed.

Usage Example: **git status**



```
MINGW64/c/Users/kmit/test
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        info.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Explanation: This command is essential to get a summary of your repository's current state. It lets you know whether files need to be added to the staging area or if there are changes to commit.

5. Git Add:

The **git add** command is used to add changes to the staging area. You must stage changes before committing them to ensure only the desired changes are included in the commit.

Usage Example: **git add .**

Explanation:

- **.** stages all changes in the current directory.
- Specific files can be added by replacing **.** with the file names.



```
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git add .
```

Example: `git add index.html`

This adds the file `index.html` to the staging area.

6. Git Commit:

The `git commit` command saves your changes to the local repository. Each commit represents a snapshot of the repository at a particular point in time. You must include a message explaining what the commit does.

Usage Example: `git commit -m "Added new feature"`

Explanation:

- `-m`: Allows you to add a commit message directly from the command line.
- The commit message should be concise but descriptive enough to explain the change.

```
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git commit -m "info file added"
[master (root-commit) cf73846] info file added
1 file changed, 3 insertions(+)
create mode 100644 info.txt
```

Example Output:

`[master (root-commit) 1a2b3c4] Added new feature`

7. Git Diff:

The `git diff` command shows the differences between files in your working directory and the staging area. It is useful to review changes before committing them.

Usage Example:

`git diff`

```

MINGW64/c/Users/kmit/test
kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git commit -m "info file added"
[master (root-commit) cf73846] info file added
1 file changed, 3 insertions(+)
create mode 100644 info.txt

kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ notepad info.txt

kmit@DESKTOP-50B7B83 MINGW64 ~/test (master)
$ git diff
diff --git a/info.txt b/info.txt
index 31c4db7..4111fbe 100644
--- a/info.txt
+++ b/info.txt
@@ -1,3 +1,6 @@
 This is my new file
 username : savram674
-email : savitharamesh674@gmail.com
 \ No newline at end of file
+email : savitharamesh674@gmail.com
+
+Department :Information Technology
+Designation: Assistant Professor
 \ No newline at end of file

```

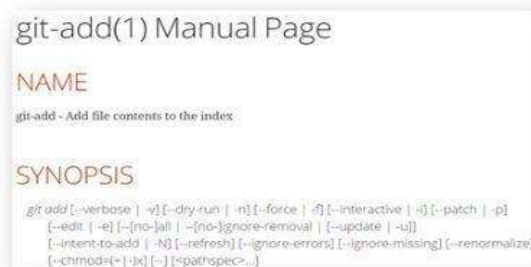
Explanation: Running **git diff** will display the differences between your modified files and the last commit, helping you see exactly what changes will be staged and committed.

8. Git Help:

The **git help** command provides documentation and detailed usage information for any Git command.

Usage Example: **git help <command>**

Explanation: You can use **git help** to get detailed information about any Git command, such as **git help add** to learn more about the **git add** command.



```

SSPL-LP-DNS-YT0+Simplilearn@SSPL-LP-DNS-YT01 MINGW64 ~
$ git help add

```

SOFTWARE ENGINEERING

UNIT – 2

TOPIC – 8

GIT COMMANDS: WORKING WITH LOCAL AND REMOTE REPOSITORIES - BRANCHES, CHECKOUT, MERGE, REVERT, LOG

Branches in Git

A **branch** in Git is like a **separate copy of your project** where you can make changes without affecting the main version. You might need different branches to work on new features, fix bugs, or prepare for a release.

- Imagine you have a main branch (called **master**) that has the latest stable version of your project. If you need to create a new feature (like adding a search function), you would create a **feature branch**. This allows you to work on the search function independently. Once the feature is complete, you can combine it (merge) with the master branch.

Types of Branches:

- **Main Branch (Master):** The main project everyone uses.
- **Feature Branch:** A branch to work on specific new features.
- **Bug Fix Branch:** Specifically for fixing bugs.
- **Release Branch:** Prepares and tests the project before it is released to users.

Important Commands:

- `git branch <branch_name>`: Creates a new branch, such as `git branch search-feature`.
- `git branch -a`: Shows a list of all branches, including the main branch, feature branches, and remote branches.

- `git branch -d <branch_name>`: Deletes a branch that you no longer need, such as `git branch -d bugfix`.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch -a
* master
  project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch -d project_changes
Deleted branch project_changes (was 4e87650).

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git branch -a
* master
```

2. Switching Between Branches: Checkout

The **git checkout** command is used to **switch between branches**. This allows you to move from one task to another easily. For example, you may be working on a new feature in one branch but need to fix a bug in another branch.

- Let's say you're working on a new search feature in a branch called `search-feature`, but suddenly a critical bug is found in the main branch (`master`). You can **switch to the bug fix branch** using `git checkout` and fix the bug. Once the bug is fixed, you can switch back to the `search-feature` branch and continue working without losing your progress.

Important Commands:

- `git checkout <branch_name>`: Switches to an existing branch, like `git checkout bugfix` to fix the bug.
- `git checkout -b <new_branch>`: Creates a new branch and switches to it immediately. For example, `git checkout -b update-design` will create and move to a new branch where you can work on a design update.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git checkout -b project_bugs
Switched to a new branch 'project_bugs'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_bugs)
$ git branch -a
master
* project_bugs
project_changes
```

3. Combining Work: Merge

After working on a separate branch for a while (e.g., developing a new feature or fixing a bug), you will want to **combine your work** with the main project. This is where **git merge** comes in—it merges the changes from one branch into another, typically merging changes into the **master** branch.

- You've finished working on the `search-feature` branch, and it's time to **merge it into the master branch**. By using `git merge search-feature` while on the master branch, you combine all the changes from the `search-feature` branch into the master. Now, the master branch contains both the original stable version and the new feature you've developed.

Important Command:

- `git merge <branch_name>`: Merges the changes from the specified branch into the current branch. For example, `git merge search-feature` combines the search feature with the master branch.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git branch project_changes

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ nano b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git add .
warning: LF will be replaced by CRLF in b.txt.
The file will have its original line endings in your working directory

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git commit -m "New file added in new branch project_changes"
[project_changes e525410] New file added in new branch project_changes
1 file changed, 1 insertion(+)
create mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ ls
a.txt b.txt
```

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (project_changes)
$ git checkout master
Switched to branch 'master'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ ls
a.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ git merge project_changes
Updating 4e87650..e525410
Fast-forward
 b.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_E
xample (master)
$ ls
a.txt b.txt
```

Activ

4. Viewing History: Log

Git keeps a **detailed history** of all changes (called **commits**) made to the project. The **git log** command shows you this history, so you can see who made changes, when they made them, and what the changes were. This is useful for tracking the progress of the project and identifying when bugs were introduced.

- Suppose you want to know when a specific feature was added or who worked on a particular part of the code. By using the `git log` command, you can see a detailed history of all the changes. Each entry in the log will show:
 - **Commit Hash:** A unique identifier for each change.
 - **Author:** The person who made the change.
 - **Date:** When the change was made.
 - **Message:** A short description of the change, like “added search functionality.”

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git log
commit e525410c637e693b0b512bbb2fe67ca8c3763b14 (HEAD -> master)
Author: Madhurika, Budaraju <budarajumadhurika@gmail.com>
Date:   Wed Oct 16 00:16:39 2024 +0530

    New file added in new branch project_changes

commit 4e87650a0f49e0b6f82b53e693099f5987eeb972 (project_bugs)
Author: Madhurika, Budaraju <budarajumadhurika@gmail.com>
Date:   Tue Oct 15 14:37:32 2024 +0530

    Created new file
```

- **Simplified View:**

If you don't want to see too much detail, you can use the `git log --oneline` command. This shows a summary of each commit, making it easier to quickly scan through the project's history.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git log --oneline
e525410 (HEAD -> master) New file added in new branch project_changes
4e87650 (project_bugs) Created new file
```

Important Commands:

- `git log`: Shows the complete history of commits in a branch.
- `git log --oneline`: Shows a simpler view with one line per commit, like `f3b9c2b`
Added search feature.

5. Undoing Changes: Revert

If a mistake was made in a previous commit, Git provides a way to **undo those changes** using the `git revert` command. This doesn't delete the commit but creates a new commit that cancels out the earlier changes. This way, the history of the project is preserved.

- Imagine you've added a new feature but later discover that it introduced a bug. Instead of deleting that commit, you can use `git revert` to undo the changes made in that commit. For example, if the commit hash for the problematic commit is `f3b9c2b`, you would use `git revert f3b9c2b`. Git will then create a new commit that undoes the changes introduced by that commit without deleting it from the project history.

Important Command:

- `git revert <commit_hash>`: Reverts (undoes) the changes from a specific commit. For example, `git revert f3b9c2b` will undo the changes made in the commit with the ID `f3b9c2b`.

```
Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (master)
$ git checkout project_changes
Switched to branch 'project_changes'

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git log --oneline
e525410 (HEAD -> project_changes, master) New file added in new branch project_changes
4e87650 (project_bugs) Created new file

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git revert e525410
[project_changes 7c2e526] Revert "New file added in new branch project_changes"
 1 file changed, 1 deletion(-)
 delete mode 100644 b.txt

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ git status
On branch project_changes
nothing to commit, working tree clean

Madhu@DESKTOP-CLQ6BHJ MINGW64 ~/Desktop/AY-23-24-Sem-2/RKR21-SE/Tesseract/Sample_Example (project_changes)
$ ls
a.txt
```

Flow of Using Git Commands (with Examples):

1. Create a Branch:

You create a branch to work on a new feature:

```
git branch new-feature
```

2. Switch to the Branch:

You switch to the new branch to start working on the feature:

```
git checkout new-feature
```

3. Develop Your Feature and Commit Changes:

You make changes, then commit them:

```
git commit -m "Developed search functionality"
```

4. Switch to Another Branch to Fix a Bug:

Suddenly, you need to fix a bug, so you switch to the bug fix branch:

```
git checkout bugfix
```

5. Merge the Bug Fix Into the Main Branch:

Once the bug is fixed, you merge it into the master branch:

```
git checkout master
```

```
git merge bugfix
```

6. Check History Using Log:

You want to review the commits to make sure everything is correct:

```
git log
```

7. Revert a Mistake:

If you discover a mistake in a previous commit, you can revert it:

```
git revert f3b9c2b
```